

Take-Home Problems

Pick **one** of the three problems below. Each one asks you to build **both sides of the system**, not just one half. Building only one half is an incomplete submission.

We evaluate you on the skills that your chosen problem exercises. Problem 1 tells us about your coding and architecture. Problem 2 tells us about your machine learning. Problem 3 tells us about your agent building.

Read this before you start

A submission that does everything asked below, works correctly, and does nothing more is a **no** from us. That is the floor, not the target.

We are a small team and every hire has to be someone the rest of us learn from. So the requirements below describe the minimum shape of a complete answer. What decides the outcome is what you do on top of it: the thing you noticed that we did not ask about, the constraint you took further than required, the failure mode you went looking for because you suspected it was there.

We are not going to tell you what that is. Finding it is the test.

The same brief goes to everyone, whether you graduate next year or have five years behind you. We do not scale the problem to your resume. Each problem has a core that a strong final-year student can complete, and a set of harder constraints at the end that most experienced engineers will not fully solve either. Go as deep as your judgement and your time allow, and be straight with us about where you stopped and why. Choosing to solve one hard constraint properly beats gesturing at four.

Two things we do not reward. Volume, because a large submission that is mostly scaffolding reads as noise. And polish over substance, because a clean interface on a system you cannot explain under questioning is the easiest thing to spot in the follow-up conversation.

Ground rules (all three problems)

Time. Budget 12 to 15 hours across 5 to 7 days. We would rather see a small system that works and is well reasoned than a large one that half runs.

AI tools. Use them. Cursor, Claude Code, Copilot, whatever you normally use. We are not testing whether you can type from memory.

But you must submit your session logs. Export the full transcript of every AI coding session and include it in your repository under `/logs`. We read these. We are looking at how you direct the tool, where you caught it being wrong, and which decisions were yours. A submission without logs is not evaluated.

Write-up. Include a `DECISIONS.md` of no more than two pages covering: the architecture you chose and what you rejected, the trade-offs you made under the time limit, how you tested it, where it breaks, and what you would do next with two more weeks.

Submission. A private GitHub repository shared with us, containing the code, the logs, the write-up, and a README that tells us how to run it in under five minutes on a clean machine.

Problem 1 — Ticket Stampede

Evaluated on: backend engineering and architecture.

Fifty thousand people want a hundred tickets, and they all arrive in the same sixty seconds.

Part A: the seller

Build a service that sells exactly one hundred tickets. Expose three endpoints.

`POST /reset` takes a ticket count and wipes all state, starting a fresh sale.

`POST /buy` takes a user id and a request id, and returns either a ticket number or a clear sold-out response.

`GET /status` returns the number sold and the full list of which user holds which ticket number.

Your service must hold four invariants under any load we throw at it. Never sell more tickets than exist. Never issue the same ticket number twice. If the same request id arrives twice, the buyer gets one ticket and not two. The count reported by `/status` always matches the tickets actually issued.

Any language, framework and datastore you like. Justify the choice in your write-up.

Part B: the buyer

Build a load client that attacks your own seller. It should be able to fire a configurable number of concurrent buy requests, replay duplicate request ids to probe your idempotency handling, and verify the four invariants afterwards by reading `/status`.

It must report requests handled per second, the median and 99th percentile response times, and a clear pass or fail on each invariant.

What we want to see

Your buyer should be able to break a naive implementation. Write the naive version first, show your client catching it overselling, then fix it. Include both the failing run and the passing run in your write-up.

Then push further. How much load does your seller take before latency degrades? Where is the bottleneck, and how do you know rather than guess? What happens when the datastore goes slow for ten seconds in the middle of the sale?

Correctness under concurrency comes first. Speed only counts once nothing is oversold.

Take it further

These are harder than the core and none of them are required. Solve one properly rather than four loosely.

Run the seller as three separate instances behind a load balancer, with no single application-level lock between them, and hold all four invariants. This is a different problem from the single-instance version, and most solutions that pass one fail the other.

Kill the datastore in the middle of a sale and bring it back. The service must not oversell, must not issue a ticket number twice, and must not lose a sale it already confirmed to a buyer. Show us the run.

Add a waitlist. When tickets run out, buyers queue. A ticket that is reserved but not confirmed within thirty seconds returns to the pool and goes to the next person in the queue. This turns a counter into a state machine and it is where most people introduce their first race condition.

On the buyer side, make the load client itself distributed across several processes and show that your throughput numbers are not being limited by your own client rather than by the seller. Prove it rather than assert it.

Problem 2 — Stump the Model

Evaluated on: machine learning and computer vision.

Someone photographs an object in the wild and wants to know exactly which catalogue item it is.

Part A: the matcher

Build a system that takes a photograph and returns the matching item from a catalogue of at least five thousand images, ranked, with the top five candidates and a confidence score.

Assemble the catalogue yourself by scraping a public source. Any product category is fine, though jewellery, watches, eyewear or footwear are closest to what we work on. Tell us what you chose and why.

We care about the retrieval approach, how you handle a photograph that has no match in the catalogue at all, and how fast a single lookup runs.

Part B: the stumper

Build the test set that tries to defeat your own matcher. Shoot at least one hundred photographs yourself, on a phone, of items that are genuinely in your catalogue. Deliberately make them hard: bad lighting, odd angles, partial occlusion, cluttered background, motion blur, reflections, a hand or a wrist in the frame.

Label each one with the correct catalogue item and with the failure conditions you were trying to induce.

Then build the evaluation harness that runs your matcher across this set and reports accuracy broken down by failure condition.

What we want to see

The interesting part is the gap between the two halves. A matcher that scores well on clean images and collapses on your own hard set is an honest and useful result, provided you diagnose why.

Define your own evaluation methodology and defend it. Tell us which failure conditions hurt most, what you tried in response, and what did not work. We would rather read a clear-eyed account of a system at seventy percent than a claim of ninety-five with no error analysis.

Take it further

These are harder than the core and none of them are required. Solve one properly rather than four loosely.

Make the system refuse correctly. Photograph twenty items that are genuinely not in your catalogue, and get your matcher to say so rather than returning its nearest wrong guess with high confidence. Report your false accept and false reject rates. This is the hardest thing on the page and almost nobody gets it right.

Grow the catalogue to a hundred thousand items while holding a lookup under one hundred milliseconds on an ordinary processor with no graphics card. Tell us what you gave up to get there.

Handle a photograph containing two or three catalogue items at once, returning a match for each rather than one confused answer.

Add a thousand new catalogue items without recomputing anything you already had, and show the accuracy on the original items is unchanged.

Automate your stumper. Instead of only shooting hard photographs by hand, generate hard cases programmatically, and show that the ones your generator produces defeat your matcher at a higher rate than your hand-shot set does.

Problem 3 — Analyst and Auditor

Evaluated on: agent design and orchestration.

A research question that no single model call can answer, and a second agent whose job is to catch the first one lying.

Part A: the analyst

Build an agent that answers open research questions requiring evidence gathered from the live web. Questions of this shape:

Which three Indian jewellery retailers opened the most new stores in the last two years, and what is your evidence for each?

List every company that raised funding in Indian quick commerce since January, with amounts, investors and dates.

For a named company, find its current head of engineering, when they joined, and where they worked before.

Your agent must plan an approach before searching, use real tools including web search and page fetching, run work in parallel where it makes sense, cross-check claims that appear in only one source, and carry what it learns across questions so that a later question about an entity it has already researched is answered faster and better.

Every answer must carry citations, and the agent must state plainly when it cannot find something rather than filling the gap with a plausible guess.

Write and run at least eight questions of your own, of increasing difficulty, with at least two that reuse entities from earlier questions so we can see the memory working.

Part B: the auditor

Build a second agent that takes an analyst answer and independently verifies it. For each claim it must open the cited source, confirm the source actually says what is claimed, and mark the claim as supported, unsupported, or contradicted. It should also flag claims with no citation at all.

Run your auditor across your own analyst's answers and report what it caught.

What we want to see

Your run logs matter as much as your answers here. For every question, we want the full trace: what the agent planned, every tool call made, what came back, and where it changed course after something failed.

Report your cost per question in tokens and rupees, and show the trend across your eight questions. An agent that gets cheaper and faster as it learns is the result we are looking for. Tell us what you changed between runs and why.

Be honest about your auditor's limits. An auditor that approves everything is telling us nothing, and we will notice.

Take it further

These are harder than the core and none of them are required. Solve one properly rather than four loosely.

Make cost fall while accuracy holds. Show your cost per question dropping by half across your eight questions with no loss in correctness, and explain exactly what mechanism produced the

drop. Caching an answer you have already seen does not count. Memory that transfers to a question you have not seen does.

Handle sources that disagree. Build in questions where two credible sources give different numbers, and make the agent resolve it and justify the choice rather than reporting both and shrugging.

Put a hard ceiling of two minutes of wall clock on each question. This forces genuine parallelism in your orchestration and exposes any agent that is really a long sequential chain.

Make the analyst adversarial. Tell it that an auditor will check every claim, then measure whether its behaviour changes. Does it cite more carefully, or does it start citing sources that look authoritative but do not support the claim? Report what you find either way.

Let the auditor's findings feed back into the analyst automatically, so the next run is better without you editing a prompt by hand. This is the closest thing on this page to what we actually build.

How we assess

First, a gate. Does it run from a clean checkout using only your README, and are the logs and write-up there? If not, we stop reading. This filters more submissions than you would expect.

Past the gate, we sort into three piles.

No. Everything asked for, correct, nothing beyond it. The write-up describes what the code does rather than why it is built that way. The session logs show the AI tool leading and the candidate accepting. No stated weaknesses, or vague ones.

Maybe. Strong on the requirements, with one or two real decisions defended well, and an honest account of at least one thing that does not work.

Yes. Something in the submission we did not ask for and did not expect, that turns out to matter. A weakness found and named before we found it. A place where the obvious approach was tried, measured, rejected with evidence, and replaced. Session logs where the candidate overrules the tool and is right to.

The session logs tell us how you work. The write-up tells us how you think. The code tells us what you can build. We weigh all three, and the write-up carries more weight than most candidates assume.

The follow-up

Everyone past the gate gets a 45-minute call. We will ask you to change your own system live, under a constraint you have not seen, using whatever tools you used to build it. This is the part that decides it, and it is why a submission you cannot fully account for is worse than a smaller one you can.

Please email your submission to careers@thuli.studio along with your linkedin profile link and a blurb about your experiences or projects in college.